

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Vnorený riadiaci systém pre model robotického manipulátora

Dokumentácia

Študijný program: Robotika a kybernetika
Predmet: I-TP-RK – Tímový projekt
Školiteľ: Ing. Martin Dodek, PhD.

**Bc. Simona Walterová, Bc. Samuel Vavruš,
Bc. Samuel Valko, Bc. Samuel Kopp,
Bc. Márk Pšenák, Bc. Filip Dubovský,
Bc. Jakub Janega**

TÍMOVÝ PROJEKT

ZADANIE

Študijný program:	Robotika a kybernetika
Študijný odbor:	kybernetika
Vedúci projektu:	Ing. Martin Dodek, PhD.
Miesto vypracovania projektu:	Ústav robotiky a kybernetiky
Riešitelia:	Bc. Simona Walterová Bc. Jakub Janega Bc. Samuel Kopp Bc. Samuel Vavruš Bc. Samuel Valko Bc. Filip Dubovský Bc. Márk Pšenák
Názov projektu:	Vnorený riadiaci systém pre model robotického manipulátora

Špecifikácia zadania:

Tímový projekt sa zameriava na implementáciu vnoreného riadiaceho systému pre model robotického manipulátora so šiestimi stupňami voľnosti. Projekt využíva mikrokontrolér STM32 a jazyk C++. Úlohou je určiť kinematickú štruktúru manipulátora na základe jeho konštrukcie (geometrie), navrhnuť a implementovať algoritmus numerického riešenia inverznej kinematickej úlohy a zabezpečiť, aby robot dokázal dosiahnuť požadovanú polohu a orientáciu efektora. Celý riadiaci algoritmus bude implementovaný ako plne vnorené riešenie na mikrokontroléri STM32, s využitím knižnice Eigen, prostriedkov operačného systému reálneho času a medziprocesovej komunikácie. Projekt zahŕňa aj návrh a implementáciu používateľského rozhrania (displej) a integráciu ovládacích prvkov (joystick a tlačidlá).

Úlohy:

1. Na základe konštrukcie modelu (geometrie) určiť kinematickú štruktúru robotického manipulátora. Formulovať doprednú a inverznú kinematickú úlohu.
2. Navrhnuť numerické riešenie inverznej kinematiky Newtonovou metódou. Implementovať algoritmus numerického riešenia inverznej kinematiky v jazyku C++ s využitím knižnice Eigen.
3. Implementovať riadiaci softvér na mikrokontroléri STM32 s využitím RTOS. Rozdeliť úlohy na procesy (výpočty, komunikácia, rozhranie) a implementovať prvky medziprocesovej komunikácie.
4. Navrhnuť užívateľské rozhranie pre ovládanie robota (joystick) a zadávanie požadovanej pozície a orientácie efektora.
5. Overiť funkčnosť a vyhodnotiť presnosť dosiahnutia polohy a orientácie.

6. Vypracovať technickú dokumentáciu k implementácii. Zdokumentovať architektúru softvéru, použité algoritmy a implementované triedy.

***Termín odovzdania projektu:* 29.5.2026**

V Bratislave dňa 1.2.2025

**prof. Ing. Jarmila Pavlovičová PhD.
garantka študijného programu**

Obsah

Úvod	6
1 Geometria modelu ramena	8
2 Priama kinematická úloha	11
2.1 Numerické riešenie	11
3 Inverzná kinematika	13
3.1 Numerické riešenie	13
3.1.1 Jakobián polohy	14
3.1.2 Jakobián orientácie	14
4 Implementácia priamej a inverznej kinematiky manipulátora	16
4.1 Priama kinematika	16
4.1.1 Pseudokód pre priamu kinematiku	16
4.2 Inverzná kinematika	17
4.2.1 Pseudokód pre Jakobián	17
4.2.2 Pseudokód pre výpočet Jakobiánu orientácie	18
4.2.3 Pseudokód pre inverznú kinematiku	18
5 Návrh používateľského rozhrania	20
5.1 Celková architektúra a princíp fungovania	20
5.2 Prostriedky používateľského rozhrania	20
5.3 Hardvérové prvky	21
5.4 Obrazovky	21
5.4.1 Obrazovka polohy XYZ	21
5.4.2 Obrazovka orientácie RPY	21
5.4.3 Obrazovka kĺbových uhlov	22
5.5 Spracovanie joysticku	22
5.6 Komunikácia s riadiacou úlohou	22
5.7 Chybové hlásenia	23
6 Overenie funkčnosti a presnosti polohovania	24
7 Ranging Sensor	25
7.1 Špecifikácia a vlastnosti senzora VL53L1X	25
7.2 Softvérová implementácia	25
7.2.1 Čo bolo implementované	25

7.3	Verejné funkcie triedy	26
7.3.1	Konštruktor	26
7.3.2	Inicializácia	26
7.3.3	Kontinuálne meranie	26
7.3.4	SD karta a logovanie	26
7.3.5	Získavanie meraní	26
7.4	Implementácia v robot_arm_servo.ino	27
7.5	Záver	27
Záver		28

Úvod

Moderná robotika sa v súčasnosti nezaobíde bez komplexných manipulátorov schopných presne, bezpečne a efektívne interagovať s okolitým prostredím. Tento tímový projekt sa zameriava na návrh a implementáciu vnoreného riadiaceho systému pre model robotického manipulátora so šiestimi stupňami voľnosti (6 DOF). Hlavnou motiváciou je vytvorenie autonómneho, stabilného a kompaktného riadiaceho riešenia, ktoré úspešne spája pokročilé kinematické výpočty s priamym nízkoúrovňovým riadením akčných členov v reálnom čase.

Pri interpretácii zadania sme sa zamerali na jednu z najdôležitejších výziev v oblasti robotiky - transformáciu požiadaviek z kartézskoho priestoru do kĺbových súradníc robota. Zatiaľ čo dopredná kinematika nám umožňuje jednoducho určiť polohu a orientáciu koncového efektora na základe známych kĺbových uhlov, pre praktické a intuitívne ovládanie je kriticky dôležité riešiť inverznú kinematickú úlohu (IKÚ). Bez nej by robotické rameno nedokázalo autonómne dosiahnuť konkrétnu cieľovú pozíciu zadanú operátorom. Keďže analytické riešenie pre 6-DOF manipulátor môže byť geometricko-matematicky zložité a často nejednoznačné, v tomto projekte implementujeme numerické riešenie IKÚ pomocou Newtonovej metódy, pričom na efektívne maticové operácie využívame optimalizovanú knižnicu Eigen.

Pre realizáciu plne vnoreného (embedded) systému bola zvolená hardvérová platforma mikrokontroléra STM32 v kombinácii s programovacím jazykom C++. Vnorené platformy sú pre riadenie robotických modelov ideálne z dôvodu ich vysokej deterministickosti, nízkej latencie, energetickej efektívnosti a možnosti priamej hardvérovej integrácie periférií. Celý riadiaci softvér je postavený na operačnom systéme reálneho času (RTOS). Tento prístup nám umožnil elegantne rozdeliť softvérovú architektúru na samostatné, paralelne bežiacie procesy - od periodických výpočtov kinematických úloh, cez robustnú medziprocesovú komunikáciu pomocou frontov správ, až po spracovanie používateľského rozhrania a samotné riadenie servomotorov.

Okrem samotného algoritmu pohybu projekt zahŕňa aj integráciu používateľského rozhrania (LCD displej a joystick s tlačidlami) na manuálne zadávanie cieľových súradníc a pokročilú nadstavbu v podobe 3D mapovania priestoru pomocou laserového senzora vzdialenosti VL53L1X s logovaním dát na SD kartu. Naším vnútorným cieľom bolo vytvoriť ucelený mechatronický systém, ktorý dokáže spoľahlivo fungovať a reagovať aj napriek fyzickým limitom a mechanickým vôľam použitého hardvéru.

Pre nás, ako študentov inžinierskeho študijného programu Robotika a kybernetika, predstavuje práca na tomto projekte neoceniteľnú skúsenosť. Ponúka nám príležitosť prejsť si celým vývojovým cyklom od analýzy geometrie fyzického modelu, cez prenos

matematických pseudokódov do produkčného C++ kódu, až po optimalizáciu a ošetrovanie numerickej stability algoritmov v singularitách pomocou regularizácie (tlmiaci parameter λ). Tento projekt je vysoko relevantný, pretože verne reflektuje reálne priemyselné výzvy v oblasti vývoja embedded softvéru a riadenia robotických systémov v reálnom čase.

1 Geometria modelu ramena

Predtým, než pristúpime k samotnej matematickej formulácii doprednej a inverznej kinematickej úlohy, je nevyhnutné bližšie predstaviť fyzickú platformu robotického ramena, ktorej vlastnosti priamo ovplyvňujú návrh riadenia. Experimentálny model manipulátora použitý v tomto projekte je koncipovaný ako malá mechatronická stavebnica. Celá mechanická konštrukcia ramena je postavená na báze cenovo dostupných komponentov a pohyb v jednotlivých kĺboch zabezpečujú štandardné modelárske servomotory. Práve pre tento konkrétny fyzický model je v nasledujúcich krokoch analyzovaná a hľadaná reálna kinematická štruktúra.

tu daj obrazok

Nasadenie tohto typu modelárskej stavebnice v rámci tímového projektu so sebou prináša špecifické vlastnosti, ktoré môžeme rozdeliť na výhody a nevýhody:

Výhody stavebnicového riešenia:

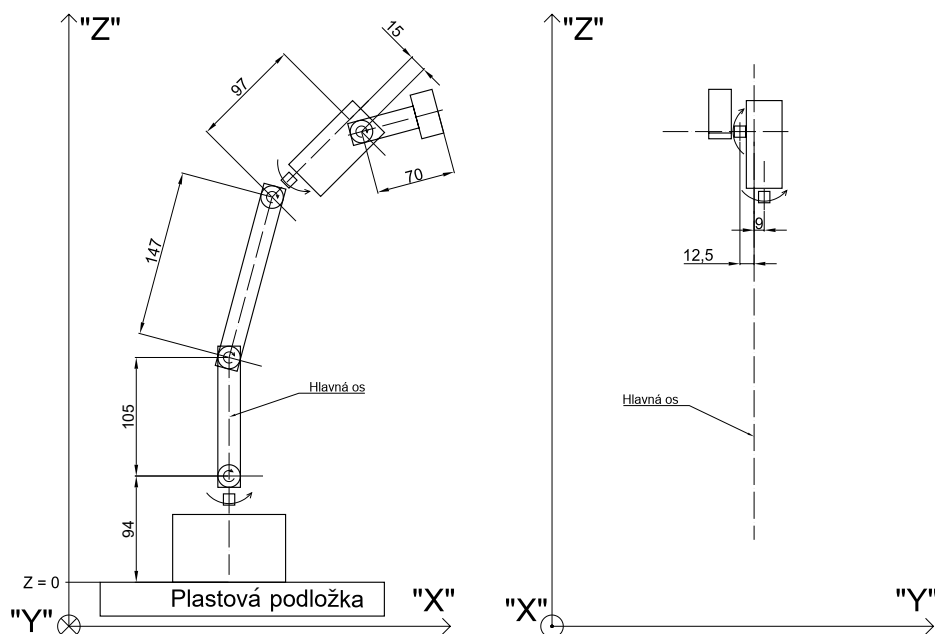
- **Vysoká dostupnosť a modularita:** Umožňuje rýchle zostavenie, prípadnú modifikáciu mechanických častí a jednoduchú výmenu komponentov v laboratórnych podmienkach.
- **Ekonomická efektivita:** Nízke obstarávacie náklady robia z platformy ideálny vzdelávací nástroj pre overenie teoretických kybernetických princípov v praxi.
- **Bezpečnosť prevádzky:** Vzhľadom na malé rozmery, nízku hmotnosť a obmedzený krútiaci moment motorov je manipulácia s modelom bezpečná a nehrozí riziko zranenia operátora ani vážneho poškodenia hardvéru pri chybách v riadiacom algoritme.

Nevýhody a mechanické limity:

- **Nízka tuhosť konštrukcie:** Spojovacie prvky vykazujú flexibilitu a konštrukcia vyrobená v domácich podmienkach nedosahuje ideálnu tuhosť profesionálnych priemyselných ramien.
- **Mechanické vôle:** Prítomnosť vôlí v prevodovkách modelárskych servomotorov a v samotných spojoch konštrukcie výrazne ovplyvňuje výslednú presnosť. Tieto vôle sa kumulujú smerom od základne k efektoru, pričom v spodných kĺboch dosahujú vôľu až 5 stupňov.
- **Obmedzená presnosť polohovania:** Použité servomotory majú praktickú presnosť pohybu na úrovni približne 3 stupňov. To spôsobuje odchýlky koncového efektora pri dosahovaní presných súradníc v kartézskom priestore.

Znalosť týchto obmedzení je kľúčová. Nelinearity, vôle a obmedzená presnosť hardvéru totiž kladú oveľa vyššie nároky na robustnosť a stabilitu softvérovej implementácie numerického riadenia, ktoré musí s týmito nepresnosťami počítať. Na základe fyzických rozmerov reálneho modelu bol vytvorený schematický náčrt, z ktorého vychádzajú všetky následné kinematické výpočty.

Na Obr. 1 je náčrt schematického zobrazenia rozmerov robotického ramena.



Obr. 1: Schematické zobrazenie rozmerov robotického ramena

Rozmery posunutí rotácie bolo kvôli implementácii výpočtov pre inverznú kinematiku nevyhnutné správne určiť. Na ľavej časti náčrtu je rameno zobrazené „z boku“, a teda os robota X smeruje vpravo a Y smerom do obrázka. V pravej časti je zobrazenie ramena „spred“, čiže os X smeruje von z obrázka a os Y smeruje vpravo. Tým, že tieto náčrty slúžia len na zdokumentovanie posunutí kĺbov (bodov otáčania) ramena, nie sú zahrnuté celkové rozmery súčiastok ramena. Taktiež v pravej časti náčrtu nie sú zobrazené prvé dva články ramena, lebo nemalo význam ich kresliť, keďže neboli v osi Y posunuté. Čiarkovaná čiara označená ako hlavná os značí smer osi Z ramena. Taktiež značí stred symetrie niektorých článkov robotického ramena, čo bolo vhodné využiť pri označovaní dĺžok posunutí kĺbov. Kruhy so šípku na koncoch článkov ramena označujú kĺby realizujúce rotáciu v osi Y a štvorce so šípku označujú kĺby konajúce rotáciu v osi Z, prípadne kĺby realizujúce rotáciu v osi Y v inom pohľade. Označenia osí v náčrtoch, ktoré v úvodzovkách neoznačujú reálne osi X, Y a Z, resp. reálny stred súradnicového systému, ale slúžia len ako pomocné osi pre lepšiu vizualizáciu smeru, z ktorého je

nákres ramena realizovaný. Skutočný počiatok súradníc robotického ramena je v bod, kde sa pretína os označená „ $Z = 0$ “ a hlavná os.

2 Priama kinematická úloha

Priama kinematická úloha (PKU) predstavuje matematický opis geometrie manipulatára, ktorý definuje zobrazenie z kĺbového priestoru do priestoru koncového efektora. Vstupom sú kĺbové premenné (uhly natočenia jednotlivých kĺbov).

Výstupom PKU je jednoznačné určenie polohy a orientácie koncového efektora (nástroja) vzhľadom na bazový (referenčný) súradnicový systém sústavy. Pre kompletný popis polohy a orientácie v 3D priestore využívame maticu homogénnej transformácie, ktorú získame postupným násobením čiastkových transformačných matíc medzi susednými kĺbovými súradnicovými systémami.

Keďže náš manipulátor sa skladá z 6 článkov, tak v nasledovných rovniciach budeme uvažovať výpočet priamej kinematickej úlohy pre týchto 6 článkov. Všeobecne však môžeme uvažovať ľubovoľný počet článkov, a teda aj ľubovoľný počet premenných.

2.1 Numerické riešenie

Pre výpočet budeme potrebovať základné vektory a matice. Budeme uvažovať polohový vektor:

$$\mathbf{p} = \begin{pmatrix} p^x \\ p^y \\ p^z \end{pmatrix} \quad (1)$$

a vektor kĺbových premenných:

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \end{pmatrix} \quad (2)$$

takzvané elementárne rotačné matice a translačný vektor na výpočet polohy koncového efektora:

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix} \quad (3)$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix} \quad (4)$$

$$R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (5)$$

$$T_x = \begin{pmatrix} L \\ 0 \\ 0 \end{pmatrix} \quad (6)$$

$$T_y = \begin{pmatrix} 0 \\ L \\ 0 \end{pmatrix} \quad (7)$$

$$T_z = \begin{pmatrix} 0 \\ 0 \\ L \end{pmatrix} \quad (8)$$

Medzi jednotlivými súradnicovými systémami platia transformácie:

$$p_5 = R_6(\theta_6)(T_6 + p_6) \quad (9)$$

$$p_4 = R_5(\theta_5)(T_5 + p_5) \quad (10)$$

$$p_3 = R_4(\theta_4)(T_4 + p_4) \quad (11)$$

$$p_2 = R_3(\theta_3)(T_3 + p_3) \quad (12)$$

$$p_1 = R_2(\theta_2)(T_2 + p_2) \quad (13)$$

$$p_0 = R_1(\theta_1)(T_1 + p_1) \quad (14)$$

Kinematika koncového efektora manipulátora je potom daná ako (transformácia z p_6 do p_0):

$$p_0(\theta) = R_1(\theta_1) \left(T_1 + R_2(\theta_2) \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \right) \quad (15)$$

Rotačné matice jednotlivých články robota tvoria celkovú rotačnú maticu pre súradnicový systém efektora, ktorú získame z funkcie priamej kinematiky. Táto matica nám umožní vypočítať orientáciu efektora vzhľadom na základňu manipulátora. Celková rotačná matica $R_T(\theta)$ je daná súčinom rotačných matíc jednotlivých článkov:

$$R_T(\theta) = R_1(\theta_1)R_2(\theta_2)R_3(\theta_3)R_4(\theta_4)R_5(\theta_5)R_6(\theta_6) \quad (16)$$

3 Inverzná kinematika

Úlohou inverznej kinematiky (IK) je nájsť také hodnoty kĺbových súradníc θ , pre ktoré koncový efektor dosiahne požadovanú polohu a orientáciu v priestore. Nech p_r predstavuje žiadanú (referenčnú) polohu a R_r žiadanú orientáciu.

Základným princípom numerického riešenia je formulácia úlohy ako problému hľadania koreňov nelineárnych rovníc. Definujeme preto reziduá (odchýlky) medzi aktuálnym a požadovaným stavom, pričom cielene hľadáme také θ , pre ktoré sa tieto vektory chyby rovnajú nule.

Reziduum (chybu) pre polohu efektora vyjadríme ako rozdiel aktuálnej a žiadanej polohy:

$$r_p(\theta) = p_0(\theta) - p_r \quad (17)$$

Náš cieľ je dosiahnuť, aby obidve reziduá boli nulové (resp. minimálne), čo znamená že polohový vektor a orientácia efektora sa zhodujú s požadovanými hodnotami p_r a $R_{RPY}(\theta_R, \theta_P, \theta_Y)$.

$$r_p(\theta) = 0 \quad (18)$$

$$r_o(\theta) = 0 \quad (19)$$

Analogicky existuje aj reziduum pre orientáciu $r_o(\theta)$, ktoré vyjadruje odchýlku aktuálnej orientácie od žiadanej orientácie R_r .

Naším cieľom je dosiahnuť, aby veľkosť chybových vektorov bola minimálna, ideálne nulová, čo znamená, že poloha aj orientácia efektora sa zhoduje s požiadavkami. Na analytický popis zmeny polohy a orientácie pri zmene kĺbových premenných a následné minimalizovanie rezidiu sa využíva Jakobián.

3.1 Numerické riešenie

Newtonova metóda bude iteratívne aktualizovať vektor kĺbových premenných θ na základe rezidiu a Jakobiánu, aby sa minimalizovali chyby polohy a orientácie efektora.

$$\left(\begin{pmatrix} \frac{\partial p_0(\theta)}{\partial \theta_1} & \frac{\partial p_0(\theta)}{\partial \theta_2} & \frac{\partial p_0(\theta)}{\partial \theta_3} & \frac{\partial p_0(\theta)}{\partial \theta_4} & \frac{\partial p_0(\theta)}{\partial \theta_5} & \frac{\partial p_0(\theta)}{\partial \theta_6} \\ (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_1} R_{RPY}^T & (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_2} R_{RPY}^T & (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_3} R_{RPY}^T & (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_4} R_{RPY}^T & (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_5} R_{RPY}^T & (0 \ 1 \ 0) \frac{\partial R_T(\theta)}{\partial \theta_6} R_{RPY}^T \\ (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_1} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_2} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_3} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_4} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_5} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_6} R_{RPY}^T \\ (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_1} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_2} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_3} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_4} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_5} R_{RPY}^T & (0 \ 0 \ 1) \frac{\partial R_T(\theta)}{\partial \theta_6} R_{RPY}^T \end{pmatrix} \right)_{\theta=\theta(k)} + A + \Delta\theta(k) = \begin{pmatrix} -r_p(\theta(k)) \\ -r_o(\theta(k)) \end{pmatrix} \quad (20)$$

$$\theta(k+1) = \theta(k) + \Delta\theta(k) \quad (21)$$

Algoritmus sa ukončí ak je splnená podmienka (veľkosť rezidua):

$$(r_p(\theta))^T(r_p(\theta)) + (r_o(\theta))^T(r_o(\theta)) < \epsilon \quad (22)$$

3.1.1 Jakobián polohy

Na výpočet inverznej kinematickej úlohy je potrebné poznať Jakobián polohy, ktorý nám umožní vypočítať zmenu polohy koncového efektora vzhľadom na zmenu kĺbových premenných. Jakobiánu polohového vektora $p_0(\theta)$ možno definovať ako maticu parciálnych derivácií $\nabla p_0(\theta) \in \mathbb{R}^{3 \times 6}$:

$$\nabla p_0(\theta) = \begin{pmatrix} \frac{\partial p_0(\theta)}{\partial \theta_1} & \frac{\partial p_0(\theta)}{\partial \theta_2} & \frac{\partial p_0(\theta)}{\partial \theta_3} & \frac{\partial p_0(\theta)}{\partial \theta_4} & \frac{\partial p_0(\theta)}{\partial \theta_5} & \frac{\partial p_0(\theta)}{\partial \theta_6} \end{pmatrix} \quad (23)$$

Na výpočet matice budeme potrebovať vypočítať parciálne derivácie polohového vektora $p_0(\theta)$ vzhľadom na jednotlivé kĺbové premenné θ_i .

$$\frac{\partial p_0(\theta)}{\partial \theta_1} = \frac{\partial(R_1(\theta_1))}{\partial \theta_1} \left(T_1 + R_2(\theta_2) \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \right) \quad (24)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_2} = R_1(\theta_1) \left(\frac{\partial(R_2(\theta_2))}{\partial \theta_2} \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \right) \quad (25)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_3} = R_1(\theta_1) R_2(\theta_2) \left(\frac{\partial(R_3(\theta_3))}{\partial \theta_3} \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \quad (26)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_4} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) \left(\frac{\partial(R_4(\theta_4))}{\partial \theta_4} \left(T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6)) \right) \right) \quad (27)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_5} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) \left(\frac{\partial(R_5(\theta_5))}{\partial \theta_5} \left(T_5 + R_6(\theta_6)(T_6) \right) \right) \quad (28)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_6} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) \frac{\partial(R_6(\theta_6))}{\partial \theta_6} (T_6) \quad (29)$$

3.1.2 Jakobián orientácie

Následne vypočítame derivácie $R_T(\theta)$ pre získanie rotačného Jakobiánu, ktorý nám umožní vypočítať zmenu orientácie efektora vzhľadom na zmenu kĺbových premenných:

$$\frac{\partial R_T(\theta)}{\partial \theta_1} = \frac{\partial R_1(\theta_1)}{\partial \theta_1} R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (30)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_2} = R_1(\theta_1) \frac{\partial R_2(\theta_2)}{\partial \theta_2} R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (31)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_3} = R_1(\theta_1) R_2(\theta_2) \frac{\partial R_3(\theta_3)}{\partial \theta_3} R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (32)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_4} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) \frac{\partial R_4(\theta_4)}{\partial \theta_4} R_5(\theta_5) R_6(\theta_6) \quad (33)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_5} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) \frac{\partial R_5(\theta_5)}{\partial \theta_5} R_6(\theta_6) \quad (34)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_6} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) \frac{\partial R_6(\theta_6)}{\partial \theta_6} \quad (35)$$

4 Implementácia priamej a inverznej kinematiky manipulátora

Riešenie vychádza z pôvodných pseudokódov, ktoré boli prenesené do programovej formy pre 6-stupňový manipulátor v C++ pod knižnicou Eigen.

Kinematický model je definovaný pomocou:

- sekvencie rotácií jednotlivých kĺbov,
- translačných vektorov medzi kĺbmi podľa mechanickej konštrukcie.

4.1 Priama kinematika

Slúži na výpočet polohy a orientácie efektora zo známych kĺbových uhlov.

Výstupom je:

- polohový vektor efektora $p \in \mathbb{R}^3$,
- rotačná matica efektora $R \in \mathbb{R}^{3 \times 3}$.

4.1.1 Pseudokód pre priamu kinematiku

Nižšie je uvedený pseudo program na výpočet priamej kinematickej úlohy, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie. Číže pseudo program vracia polohový vektor a rotačnú maticu koncového efektora vzhľadom na základňu manipulátora.

```
function forwardKinematics(theta[], R[])
    vekt = 0;
    matica = I_3x3;
    n = size(theta[]);

    for (i = n; i > 0; i--) begin
        vekt = R[i](theta[i]) * (T[i] + vekt);
        matica = R[i](theta[i]) * matica;
    end

    return vekt, matica;
end function
```

Výpis kódu 1: Výpočet priamej kinematickej úlohy

Poloha je počítaná rekurzívne od konca článku smerom k základni. Orientácia efektora vzniká ako súčin rotačných matíc všetkých kĺbov v poradí od základne po efektor.

4.2 Inverzná kinematika

Hľadá požadované kĺbové uhly iteratívnou Newtonovou metódou. Cieľom je nájsť také kĺbové uhly, aby efektor dosiahol požadovanú polohu a orientáciu.

- **Výpočet chyby:** V každom kroku sa určuje polohová a orientačná chyba (reziduum) oproti požadovanému stavu.
- **Riešenie sústavy:** Zmena kĺbových uhlov sa počíta cez úplný Jakobián riešením lineárnej sústavy $J\Delta\theta = -r$, kde r je spojený vektor chýb.
- **Regularizácia:** Na zabezpečenie numerickej stability pri horšej podmienenosti Jakobiánu a v singularitách sa používa tlmiaci parameter λ .
- **Normalizácia:** Po každej iterácii sa vypočítané uhly zarovnávajú do intervalu $[-\pi, \pi]$, čím sa bráni ich nekončenému pretáčaniu.

4.2.1 Pseudokód pre Jakobián

Nižšie je uvedený pseudo program na výpočet Jakobiho matice, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie. Tento pseudo program nám umožní vypočítať Jakobián polohy efektora vzhľadom na kĺbové premenné.

```
function jacobian(theta[], R[], dR[])
    n = size(theta[]);
    jacob_mat = 3 x n;
    matica = I_3x3;
    pom_vekt = T[n];

    for(i = n-1; i > 0; i--)
        begin
            matica = R[i](theta[i]) * matica;
        end

    for(j = n; j > 0; j--)
        begin
            vekt = T[n];
            if(n > j)
                begin
                    matica = matica * R[j](theta[j]) * transpose();
                    pom_vekt = T[j] + R[j+1](theta[j+1]) * pom_vekt;
                    vekt = pom_vekt;
                end

            vekt = matica + dR[j](theta[j]) * vekt;
            jacob_mat.col(j) = vekt;
        end

    return jacob_mat;
end function
```

Výpis kódu 2: Výpočet Jakobiho matice

4.2.2 Pseudokód pre výpočet Jakobiánu orientácie

Nižšie je uvedený pseudo program na výpočet derivácií rotačných matíc, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie.

```
function derRotMat(theta[], dR[], R[])
    n = size(theta[]);
    mat_all(n);
    f_mat = I_3x3;
    b_mat = f_mat;

    for(i = n-1; i > 0; i--)
        begin
            f_mat = R[i](theta[i]) * matica;
        end

    for(j = n; j > 0; j--)
        begin
            if(n > j)
                begin
                    b_mat = R[j+1](theta[j+1]) * b_mat;
                    f_mat = f_mat * R[j](theta[j]).transpose();
                end
            mat_all(j) = f_mat * dR[j](theta[j]) * b_mat;
        end

    return mat_all;
end function
```

Výpis kódu 3: Výpočet derivácií rotačných matíc

4.2.3 Pseudokód pre inverznú kinematiku

Nižšie je uvedený pseudo program na iteratívny výpočet inverznej kinematiky pomocou Newtonovej metódy, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie.

```
function solveIK(pr, Rr, theta, tol_pos, tol_ori, max_iter)
    for(i=0; i < max_iter; i++)
        begin
            [p0, R0] = forwardKinematics(theta, R);
            (3,3) rp = p0 - Pr;
            (3,3) E = R0 + Rr.transpose();
            ro(0,0) = E(1,0); // (010) * E(100) ~T
            ro(1,0) = E(2,0); // (001) * E(100) ~T
            ro(2,0) = E(2,1); // (001) * E(010) ~T

            if(rp.norm < tol_pos && ro.norm < tol_ori)
                begin
                    return 1; // úspešne max_iterov
                end

            J_p = jacobian(theta, R, dR);
            der_R = derRotMat(theta, dR, R);
```

```

J.block<3,6>(0,0) = J_p.block<3,6>(0,0);

for(j=0; j < DOF; ++j)
begin
(3,3) dE_j = der_R[j] * Rr.transpose();

J(3,j) = dE_j(1,0);
J(4,j) = dE_j(2,0);
J(5,j) = dE_j(2,1);
end

for(k=0; k < DOF; ++k)
begin
    J(k,k) += lambda;
end

minus_r.block<3,1>(0,0) = -rp;
minus_r.block<3,1>(3,0) = -ro;
delta_theta = vyries_sustavu_6x6(J, minus_r);
theta = theta + delta_theta;
end
return false;
end function

```

Výpis kódu 4: Iteratívny výpočet inverznej kinematiky

Podmienky ukončenia a robustnosť

Iterácie sa ukončia pri splnení oboch podmienok:

- polohová chyba je menšia ako zadaná tolerancia,
- orientačná chyba je menšia ako zadaná tolerancia.

Ak sa riešenie nenájde v definovanom maximálnom počte iterácií, stav sa vyhodnotí ako nekonvergentný. Tento mechanizmus chráni systém pred nekonečnou iteráciou pri nedosiahnuteľných cieľoch.

Prevádzka v reálnom čase

IK beží v periodickej riadiacej slučke. Táto slučka neustále prijíma požadovanú pozíciu, prepočítava ju na nové uhly, overuje fyzické limity pohonov a až po ich validácii odošle dáta do servopohonov. Tým je zaistený plynulý a bezpečný pohyb ramena.

5 Návrh používateľského rozhrania

5.1 Celková architektúra a princíp fungovania

Z hľadiska celkovej architektúry je riadenie robotického ramena implementované na vývojovej doske s mikrokontrolérom STM32. O spoľahlivý beh jednotlivých procesov sa stará operačný systém reálneho času FreeRTOS. Softvér je logicky rozdelený na nezávislé, no navzájom komunikujúce úlohy (tasky), pričom kľúčové sú najmä:

- **Riadiaci a kinematický task (Control Task)** – beží v striktne vyhradenej periodickej slučke. Zabezpečuje prepočet požadovanej polohy a orientácie na kĺbové súradnice pomocou inverznej kinematiky (IK), overuje fyzické limity servomotorov a stará sa o generovanie plynulej trajektórie. Následne odosiela povelý priamo do servopohonov.
- **Task používateľského rozhrania (UI Task)** – stará sa o čítanie vstupov (joystick, tlačidlá), obsluhu displeja a spracovanie volieb používateľa, ktoré obratom posiela do riadiaceho tasku.

Očakávané správanie systému z používateľského hľadiska je také, že rameno po spustení funguje ako samostatný vstavaný systém (embedded device) bez nutnosti stáleho pripojenia k počítaču. Používateľ prostredníctvom ovládacieho panelu volí režim riadenia a plynule zadáva cieľovú pozíciu koncového efektora (X, Y, Z súradnice), jeho orientáciu (Roll, Pitch, Yaw), alebo natáča jednotlivé kĺby priamo. Systém následne automaticky naplánuje pohyb a manipulátor sa plynule, bezpečne a bez kolízií presunie do požadovaného stavu.

5.2 Prostriedky používateľského rozhrania

Používateľské rozhranie manipulátora slúži na samotné zadávanie požadovanej polohy, orientácie alebo priamo kĺbových uhlov robota. Rozhranie je realizované pomocou LCD displeja 16x2 s I2C prevodníkom a joysticku s tlačidlami.

Systém beží ako samostatná FreeRTOS úloha **TaskUI**, ktorá zabezpečuje:

- čítanie vstupu z joysticku,
- prepínanie medzi obrazovkami,
- prepínanie medzi módmi ovládania,
- zobrazovanie aktuálneho stavu robota,
- odosielanie požiadaviek do riadiacej úlohy.

5.3 Hardvérové prvky

Používateľské rozhranie využíva nasledovné hardvérové prvky:

- **Joystick** – slúži na zmenu hodnôt v dvoch osiach.
- **Tlačidlo A** – prepína medzi jednotlivými obrazovkami.
- **Tlačidlo B** – prepína medzi módmi v rámci obrazovky.
- **LCD 16x2** – zobrazuje aktuálne hodnoty.

5.4 Obrazovky

Používateľské rozhranie obsahuje tri základné obrazovky:

5.4.1 Obrazovka polohy XYZ

Na tejto obrazovke sa nastavuje poloha koncového efektora v priestore.

```
X:12.4  Y:8.1  
Z:4.2   XY
```

Výpis kódu 5: Ukážka obrazovky XYZ

V pravom dolnom rohu sa zobrazuje aktuálny mód:

- **XY** – joystick ovláda osi X a Y
- **YZ** – joystick ovláda osi Y a Z
- **ZX** – joystick ovláda osi Z a X

5.4.2 Obrazovka orientácie RPY

Na tejto obrazovke sa nastavuje orientácia efektora pomocou Roll, Pitch a Yaw uhlov.

```
R:15.0  P:8.2  
Y:4.1   RP
```

Výpis kódu 6: Ukážka obrazovky RPY

Možné módy:

- **RP** – joystick ovláda Roll a Pitch
- **PY** – joystick ovláda Pitch a Yaw
- **YR** – joystick ovláda Yaw a Roll

5.4.3 Obrazovka kĺbových uhlov

Na tejto obrazovke sa nastavujú jednotlivé uhly servomotorov.

T1:20.0 T2:10.0
PAIR:12

Výpis kódu 7: Ukážka obrazovky JOINTS

Možné módy:

- PAIR:12 – joystick ovláda θ_1 a θ_2
- PAIR:34 – joystick ovláda θ_3 a θ_4
- PAIR:56 – joystick ovláda θ_5 a θ_6

5.5 Spracovanie joysticku

Joystick poskytuje analógové hodnoty v rozsahu približne 0 až 1023. Stredová hodnota je približne:

$$CENTER = 520 \quad (36)$$

Kvôli odstráneniu šumu sa používa mŕtva zóna:

$$DEADZONE = 20 \quad (37)$$

Výpočet zmeny:

$$dx = (joyX - CENTER) \cdot k \quad (38)$$

$$dy = (joyY - CENTER) \cdot k \quad (39)$$

kde k je koeficient citlivosti.

5.6 Komunikácia s riadiacou úlohou

UI komunikuje s riadiacou úlohou pomocou FreeRTOS front:

- gUiToControlQueue – odosielanie požadovanej polohy/orientácie/kĺbových uhlov
- gControlToUiQueue – prijímanie aktuálneho stavu robota

Ak je robot v režime polohy/orientácie:

$$[x, y, z, R, P, Y] \quad (40)$$

sa odošlú do inverznej kinematiky.

Ak je robot v režime kĺbového ovládania:

$$[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6] \quad (41)$$

sa odošlú priamo regulátoru servomotorov.

5.7 Chybové hlásenia

Pri výskyte chyby sa na displeji zobrazí chybová správa.

CONTROL ERROR
IK FAILED

Výpis kódu 8: Ukážka chyby

alebo

CONTROL ERROR
OUT OF LIMITS

Výpis kódu 9: Ukážka chyby

Týmto spôsobom používateľ okamžite vidí, že požadovaná poloha nie je dosiahnuteľná alebo prekračuje limity robota.

6 Overenie funkčnosti a presnosti polohovania

Napriek vypracovanému softvéru sa pri overovaní funkčnosti zistili určité obmedzenia presnosti. Servomotory, ktoré tvoria základný pohybový systém ramena, dosahujú praktickú presnosť pohybu na úrovni 3 stupňov, pričom vo spodných kĺboch je táto voľa ešte vyššia, až 5 stupňov. Táto nepresnosť je spôsobená viacerými faktormi, ktoré sú ťažko zvládnuteľné bez výrazného zvýšenia nákladov.

Primárnym zdrojom nepresnosti je mechanická vôľa a pružnosť v konštrukcii ramena. Spojovacie prvky a prenosy pohybu medzi jednotlivými kĺbmi nemôžu byť s dokonalou tuhosťou vyrobené v domácich podmienkach. Okrem toho, servomotory a ich riadiaca elektronika majú limity presnosti. Tieto motory boli vybrané na základe dostupnosti a ceny, nie na základe špecifikácie vysokej presnosti, čo je prirodzeným kompromisom pri projektovaní vzdelávacieho robota.

Napriek týmto obmedzeniam je robot plne funkčný a splňa účel vzdelávacieho nástroja. Presnosť pohybu je dostatočná na demonštráciu robotických princípov a základných algoritmov riadenia. Pre aplikácie vyžadujúce vyššiu presnosť by bolo potrebné použiť profesionálnejšie časti s presnejšími servomotormi a lepšou mechanickou konštrukciou.

7 Ranging Sensor

Súčasťou výbavy manipulátora je dištančný snímač (angl. Ranging Sensor), ktorý plní funkciu bezkontaktného merania vzdialenosti k objektom v pracovnom priestore. Na tento účel bol zvolený senzor **VL53L1X** od spoločnosti STMicroelectronics.

7.1 Špecifikácia a vlastnosti senzora VL53L1X

Senzor VL53L1X funguje na pokročilom princípe Time-of-Flight (ToF). Vyžaruje neviditeľný infračervený laserový lúč a meria absolútny čas letu fotónov (od vyslania po návrat), čím poskytuje presnú vzdialenosť úplne nezávisle od farby či povrchovej odrazivosti prekážky. Medzi jeho kľúčové vlastnosti patrí:

- Maximálny dosah snímania až do 4 metrov v závislosti od podmienok a režimu.
- Tri prepínateľné módy (Short, Medium, Long), ktoré dovoľujú optimalizovať senzor na kratšie, no rýchlejšie merania, alebo pomalšie merania do väčších vzdialeností.
- Úsporná a jednoduchá sériová komunikácia prostredníctvom I2C zbernice.

Snímač je fyzicky upevnený priamo na **koncovom efektore** robotického ramena. Vďaka tomuto umiestneniu na samom konci kinematického reťazca sa zorné pole senzora posúva a natáča spolu s efektorom. To otvára možnosti pre dynamické merania, napríklad 3D mapovanie pracovného priestoru pripojením nameranej vzdialenosti k aktuálnym kĺbovým a polohovým súradniciam ramena.

7.2 Softvérová implementácia

Nižšie je uvedený prehľad implementácie triedy **RangingSensor**, ktorá slúži ako konfiguračný a potvrdzovací wrapper práve pre modul VL53L1X, pričom rozširuje jeho základnú funkcionálnosť o priestorové mapovanie a logovanie dát na SD kartu.

7.2.1 Čo bolo implementované

- Wrapper trieda pre senzor VL53L1X.
- Podpora 3D mapovania priestoru.
- Štruktúra SensorPose (x, y, z, roll, pitch, yaw).
- Štruktúra MeasurementData (meranie + aktuálna poloha).
- SPI inicializácia SD karty.
- Zápis nameraných dát do súboru octomap.txt pre neskoršiu rekonštrukciu mapy.

7.3 Verejné funkcie triedy

7.3.1 Konštruktor

```
RangingSensor(TwoWire& wire)
```

Pripraví inštanciu senzora naviazanú na konkrétnu I2C zbernicu (napr. Wire3).

7.3.2 Inicializácia

```
bool init(uint32_t sample_period_ms, DistanceMode mode)
```

Inicializuje senzor, nastaví časový rozpočet a režim merania.

- `sample_period_ms` – perióda merania v milisekundách.
- `mode` – režim senzora: Short / Medium / Long.
- Návratová hodnota: true pri úspechu, false pri chybe.

7.3.3 Kontinuálne meranie

```
void startContinuous() / void stopContinuous()
```

Spustenie alebo zastavenie kontinuálneho snímania vzdialenosti.

7.3.4 SD karta a logovanie

```
bool initSD(uint32_t cs_pin)
```

Inicializuje SD kartu cez SPI rozhranie.

```
void enableLogging(bool enable, bool clear = false)
```

Zapína alebo vypína zápis do súboru octomap.txt. Obsahuje buffer manažment (`flush_counter`).

- `enable` – true zapne logovanie, false vypne logovanie.
- `clear` – true vymaže predchádzajúci súbor.

7.3.5 Získavanie meraní

```
MeasurementData read(float x, float y, float z, float roll, float pitch,  
float yaw)
```

Načítá najnovšiu vzdialenosť a priradí k nej aktuálnu polohu/orientáciu senzora. Pri zapnutom logovaní automaticky prepočíta lúč do 3D tela a uloží výsledok na SD kartu.

7.4 Implementácia v robot_arm_servo.ino

1. Vytvorenie objektu: `RangingSensor rangingSensor(Wire3);`

2. Inicializácia senzora v `setup()`:

```
rangingSensor.init(Ts_ms, RangingSensor::DistanceMode::Short);
```

3. Inicializácia SD karty: `rangingSensor.initSD(PE4);`

4. Zapnutie logovania a vymazanie starých dát:

```
rangingSensor.enableLogging(true, true);
```

5. Spustenie kontinuálneho režimu: `rangingSensor.startContinuous();`

6. V hlavnej slučke čítanie meraní:

```
MeasurementData measurement = rangingSensor.read(x, y, z, roll, pitch,  
yaw);
```

7.5 Záver

Trieda `RangingSensor` zabezpečuje kompletnú integráciu senzora VL53L1X do robotického ramena. Spája meranie vzdialenosti, informácie o polohe a orientácii ramena a vytvárajúci dátový základ pre 3D rekonštrukciu priestoru.

Záver

Záver (ak je potrebný)